Shoeb Sheikh CS23074

Lab 7 Extra Practical: To apply Support Vector Machine for Regression on Students Performance Dataset and analyze the impact of different kernels on model accuracy.

```
import pandas as pd
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

```
path= "/content/drive/MyDrive/ML_Classes/Student_Performance_dataset_5000.csv"
df=pd.read_csv(path)
df.head (5)
```

	student_id	age	gender	academic_level	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_
0	1	18	Other	High School	7.64	1.56	2.20	3.05	
1	2	18	Other	High School	2.21	2.22	2.10	1.65	
2	3	22	Male	High School	3.45	0.00	0.29	1.34	
3	4	17	Other	High School	5.75	2.08	3.01	2.27	
4	5	19	Other	High School	6.83	1.72	3.33	2.65	

5 rows × 21 columns

```
df.describe()
```

	student_id	age	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours	sleep
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.
mean	2500.500000	20.520400	4.539594	2.478734	2.011984	2.998086	1.564514	7.
std	1443.520003	2.870406	1.821665	1.177990	0.983906	1.467949	1.110807	1.
min	1.000000	16.000000	0.000000	0.000000	0.000000	0.000000	0.000000	4.
25%	1250.750000	18.000000	3.250000	1.660000	1.320000	1.990000	0.670000	6.
50%	2500.500000	20.000000	4.530000	2.480000	2.010000	2.980000	1.490000	7.
75%	3750.250000	23.000000	5.760000	3.290000	2.690000	4.030000	2.340000	7.
max	5000.000000	25.000000	11.840000	7.410000	6.000000	8.280000	5.640000	10.

```
df.shape
```

```
(5000, 21)
```

```
# Remove leading and trailing spaces from column names
df.columns = df.columns.str.strip()
print(df.columns)
```

```
Index(['student_id', 'age', 'gender', 'academic_level', 'study_hours',
       'self_study_hours', 'online_classes_hours', 'social_media_hours',
       'gaming_hours', 'sleep_hours', 'screen_time_hours', 'exercise_minutes',
       'caffeine_intake_mg', 'part_time_job', 'upcoming_deadline',
       'internet_quality', 'mental_health_score', 'focus_index',
       'burnout_level', 'productivity_score', 'exam_score'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 21 columns):
```

```
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   student_id            5000 non-null   int64
 1   age                   5000 non-null   int64
 2   gender                5000 non-null   object
 3   academic_level        5000 non-null   object
 4   study_hours           5000 non-null   float64
 5   self_study_hours      5000 non-null   float64
 6   online_classes_hours  5000 non-null   float64
 7   social_media_hours    5000 non-null   float64
 8   gaming_hours          5000 non-null   float64
 9   sleep_hours           5000 non-null   float64
 10  screen_time_hours     5000 non-null   float64
 11  exercise_minutes      5000 non-null   int64
 12  caffeine_intake_mg    5000 non-null   int64
 13  part_time_job         5000 non-null   int64
 14  upcoming_deadline     5000 non-null   int64
 15  internet_quality      5000 non-null   object
 16  mental_health_score   5000 non-null   int64
 17  focus_index           5000 non-null   float64
 18  burnout_level         5000 non-null   float64
 19  productivity_score    5000 non-null   float64
 20  exam_score            5000 non-null   float64
dtypes: float64(11), int64(7), object(3)
memory usage: 820.4+ KB
```

```
if 'student_id' in df.columns:
    df = df.drop(columns=['student_id'])
```

```
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

```
X.head()
```

	age	gender	academic_level	study_hours	self_study_hours	online_classes_hours	social_media_hours	gaming_hours	sleep_
0	18	Other	High School	7.64	1.56	2.20	3.05	2.19	
1	18	Other	High School	2.21	2.22	2.10	1.65	2.55	
2	22	Male	High School	3.45	0.00	0.29	1.34	2.08	
3	17	Other	High School	5.75	2.08	3.01	2.27	2.20	
4	19	Other	High School	6.83	1.72	3.33	2.65	0.70	

```
X = pd.get_dummies(X, drop_first=True)
```

```
df.columns
```

```
Index(['age', 'gender', 'academic_level', 'study_hours', 'self_study_hours',
       'online_classes_hours', 'social_media_hours', 'gaming_hours',
       'sleep_hours', 'screen_time_hours', 'exercise_minutes',
       'caffeine_intake_mg', 'part_time_job', 'upcoming_deadline',
       'internet_quality', 'mental_health_score', 'focus_index',
       'burnout_level', 'productivity_score', 'exam_score'],
      dtype='object')
```

```
if y.dtype != 'int64' and y.dtype != 'int32':
    median_value = y.median()
    y = (y >= median_value).astype(int)
    print("\nTarget converted into Binary (Above/Below Median).")
print("\nFinal Feature Shape:", X.shape)
```

```
Final Feature Shape: (5000, 22)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
y_train
```

	exam_score
1840	10.53
2115	35.73
4437	25.82
1146	20.95
2486	27.36
...	...
4426	10.57
466	23.77
3092	25.68
3772	17.15
860	2.81

3500 rows × 1 columns

dtype: float64

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
results = {}
```

```
svc_default = SVC(C=100.0)
svc_default.fit(X_train, y_train)
y_pred_default = svc_default.predict(X_test)
results["Default RBF"] = accuracy_score(y_test, y_pred_default)
print
('Model accuracy score with RBF kernel and c=100.0 : {0:.4f}'.format(results["Default RBF"]))
```

```
'Model accuracy score with RBF kernel and c=100.0 : 0.8193'
```

```
svc_linear = SVC(kernel='linear', C=1.0)
svc_linear.fit(X_train, y_train)
y_pred_linear = svc_linear.predict(X_test)
results["Linear"] = accuracy_score(y_test, y_pred_linear)
print
('Model accuracy score with linear kernel and c=1.0: {0:.4f}'.format(results["Linear"]))
```

```
'Model accuracy score with linear kernel and c=1.0: 0.8720'
```

```
svc_poly = SVC(kernel='poly', C=1.0)
svc_poly.fit(X_train, y_train)
y_pred_poly = svc_poly.predict(X_test)
results["Polynomial"] = accuracy_score(y_test, y_pred_poly)
```

```
svc_sigmoid = SVC(kernel='sigmoid', C=1.0)
svc_sigmoid.fit(X_train, y_train)
y_pred_sigmoid = svc_sigmoid.predict(X_test)
results["Sigmoid"] = accuracy_score(y_test, y_pred_sigmoid)
```

```
print("\nModel Accuracy Comparison:")
for model, acc in results.items():
    print(f"{model}: {acc:.4f}")
```

```
Model Accuracy Comparison:
Default RBF: 0.8193
RBF (C=100): 0.8193
Linear: 0.8720
Polynomial: 0.8560
Sigmoid: 0.8647
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred_linear) # Using predictions from the best performing linear model
```

```
print(cm)
```

```
Confusion Matrix:
[[664 100]
 [ 92 644]]
```